TS-PC-01 — Autentizace hráče (Registrace a Přihlášení)

Metoda: Task-Based Usability Test + Security & State Check

Testovaný princip: Srozumitelnost registračního formuláře, frontend validace vstupů, bezpečné odeslání dat (Network) a správa uživatelské relace (Session/Token perzistence).

Cíl scénáře: Ověřit, zda uživatel dokáže plynule vytvořit nový účet a přihlásit se do něj. Z technického hlediska zkontrolovat, jak aplikace ošetřuje chybné nebo duplicitní vstupy, minimalizuje zbytečné API requesty a jak udržuje uživatele přihlášeného i po obnovení stránky.

Příprava:

- Aplikace je odhlášená.
- Připravené testovací údaje: jeden zcela nový e-mail a jeden e-mail, který už je v databázi prokazatelně zaregistrovaný.
- Otevřené DevTools (ideálně záložky *Console*, *Network* a *Application*).

Situace pro účastníka:

"Chceš si svůj postup ve hře ukládat do cloudu, abys o něj nepřišel a mohl hrát i z mobilu. Potřebuješ si vytvořit herní účet a následně se do něj přihlásit."

Testovací kroky:

#	Akce	Instrukce	Kritérium splnění (UX + Systém)
1	**Orientace a navigace**	"Kde bys začal, když si chceš vytvořit nový účet?"	**UX:** Uživatel okamžitě najde tlačítko Registrace / Vytvořit účet. **Systém:** Formulář se vykreslí bez chyb v konzoli.
2	**Frontend validace (Edge case)**	"Zkus do hesla napsat jen '123' a jméno. Pak zkus formulář odeslat."	**UX:** Tlačítko se buď nezpřístupní, nebo vyskočí jasná, česky psaná chybová hláška (např. "Heslo musí mít alespoň 6 znaků"). **Systém:** V záložce *Network* se objeví request se statusem 400
3	**Úspěšná registrace a API call**	"Teď to oprav, zadej platné jméno a silné heslo a zaregistruj se."	**UX:** Uživatel dostane pozitivní feedback (např. "Účet vytvořen" nebo je rovnou přesměrován k přihlášení/do hry). **Systém:** V záložce *Network* se objeví request se statusem 200

4	**Ošetření duplicity na backendu**	"Zkus si teď schválně vytvořit úplně nový účet, ale použij stejný e-mail jako před chvílí."	**UX:** Aplikace nespadne, ale vypíše uživatelsky přívětivou chybu ("Tato přeydívka je již obsazena"). **Systém:** Server vrátil chybu (např. `409 Conflict` nebo `400 Bad Request`) a frontend ji správně zpracoval a zobrazil, aniž by vyhodil unhandled exception v konzoli.
5	**Přihlášení a Session Management**	"Běž na přihlášení a přihlas se údaji z prvního pokusu. Jak poznáš, že jsi uvnitř?"	**UX:** Rozhraní se změní (objeví se jméno uživatele, tlačítko Odhlásit atd.). **Systém:** Po úspěšném loginu server vrátil autorizační token (např. JWT) nebo session ID, které se bezpečně uložilo do *Local Storage* nebo *Cookies* (ověřeno v DevTools -> Application).
6	**Test perzistence (Hard Refresh)**	"Zkus teď natvrdo obnovit celou stránku (F5)."	**UX:** Uživatel zůstává přihlášený, nevyhodilo ho to zpět na logovací obrazovku. **Systém:** Aplikace si při initial loadu úspěšně přečetla token/cookie a obnovila stav přihlášeného uživatele (state).

Chybějící funkcionalita / Chybové stavy k reportování:

- Pokud formulář nehlásí vizuálně chyby u špatně vyplněných polí:
  - **MantisBT:** "Chybí frontendová validace registračního formuláře" -> Severity = major, Priority = high.
- Pokud uživatel po stisknutí F5 vypadne ze svého účtu:
  - **MantisBT:** "Nefunguje perzistence relace, uživatel je po refreshi odhlášen" -> Severity = major, Priority = high.
- *(Volitelně pro hlubší kontrolu)* Pokud se heslo při registraci posílá v URL místo v těle POST requestu:
  - **MantisBT:** "Bezpečnostní riziko: Heslo je odesíláno v URL (GET request) místo POST payloadu" -> Severity = block, Priority = immediate.

Závěrečné otázky:

- "Bylo ti u registrace jasné, jaká pravidla pro heslo hra vyžaduje?"
- "Přišlo ti střídání obrazovek mezi registrací a přihlášením plynulé a logické?"